

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Объектно-ориентированное программирование»

Студент гр. 4385

Юферев А.С.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2025

Цель работы:

Реализация системы сохранения и загрузки состояния игры с использованием механизмов сериализации, обработка исключительных ситуаций при работе с файловой системой, а также расширение пользовательского интерфейса для управления сохранениями.

Задача:

В рамках лабораторной работы необходимо расширить существующую игру следующими возможностями:

1. Создание меню с выбором действий (новая игра, загрузка, выход).
2. Реализация системы сохранения текущего состояния игры в файл.
3. Реализация системы загрузки игры из файла.
4. Обработка исключительных ситуаций при работе с файлами:
 - отсутствие файла сохранения,
 - повреждённые данные,
 - ошибки записи/чтения.
5. Обеспечение консистентности сохранений между запусками игры.
6. Сохранение всех значимых параметров игры: размеры поля, состояние игрока, список заклинаний, позиции и здоровье врагов.

Выполнение работы:

1. Разделение ответственности

В существующей игре класс Game выполнял слишком много функций: управлял игровым циклом, обрабатывал ввод, содержал логику победы/поражения. Для реализации сохранений было принято решение создать новые классы, каждый из которых отвечает за строго определённую функциональность:

- SaveManager — отвечает исключительно за операции с файловой системой (создание папки, запись, чтение, список файлов).
- StartMenu — управляет главным меню и выбором пользователя.
- SaveLoadMenu — отображает список доступных сохранений и обрабатывает выбор.

- `GameApplication` — координирует работу меню и запуск игры.

2. Формат сохранения

Выбран текстовый формат с маркерами секций

([FIELD], [PLAYER], [SPELLS], [ENEMIES]). Такой подход обеспечивает:

- Человеко-читаемость — можно открыть файл и проверить корректность данных.
- Устойчивость к изменениям — при добавлении новых полей старые сохранения не ломаются.
- Простоту отладки — легко понять структуру данных.

3. Обработка исключений

Создана иерархия классов исключений:

- `SaveException` — базовый класс для ошибок сохранения.
- `LoadException` — базовый класс для ошибок загрузки.

Это позволяет разделить логику обработки разных типов ошибок и давать пользователю понятные сообщения.

4. Интеграция с существующим кодом

Для минимизации изменений в работающей игре были добавлены только необходимые методы:

- `getSaveData()` — сбор текущего состояния в структуру.
- `loadFromSave()` — восстановление состояния из структуры.
- `saveToFile()` — обработчик сохранения по клавише.

Существующая игровая логика не изменялась, что гарантирует сохранение работоспособности.

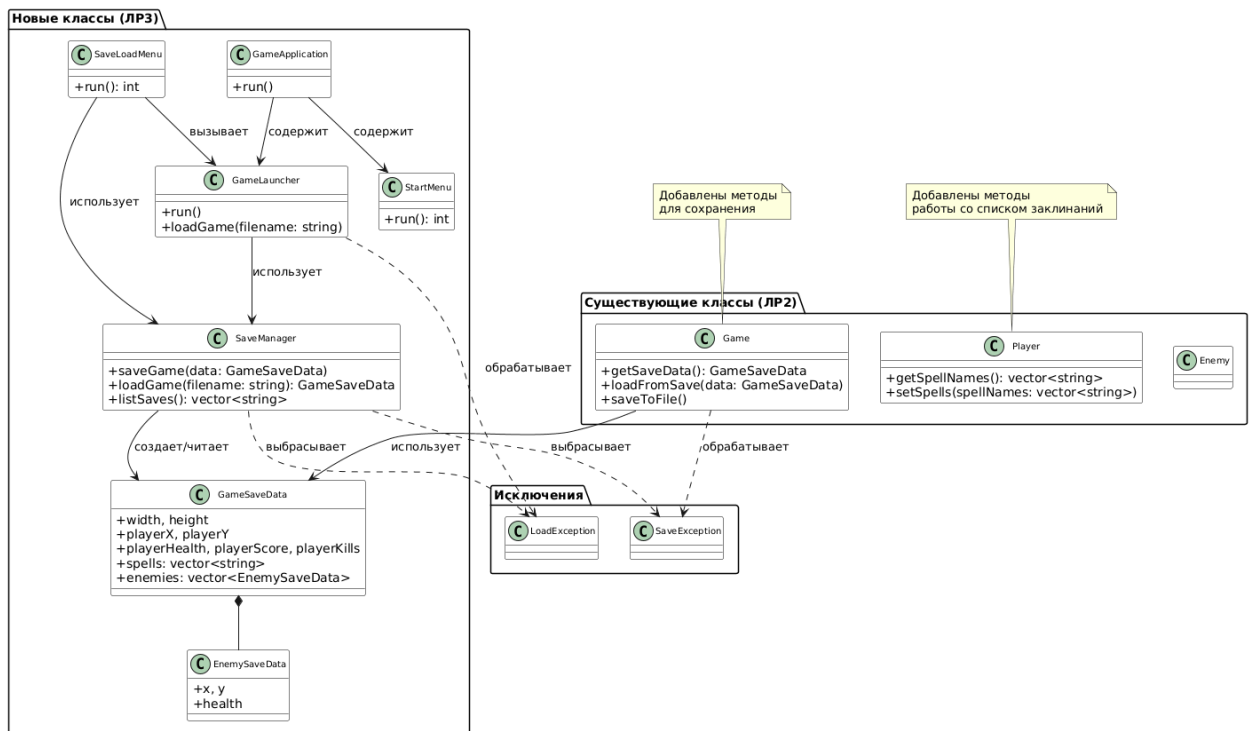


Рисунок 1 – UML-диаграмма классов игры

Описание классов:

SaveManager:

Назначение: централизованное управление операциями сохранения и загрузки.

Структуры GameSaveData и EnemySaveData:

Назначение: контейнеры для передачи состояния между игрой и менеджером сохранений.

Классы исключений:

Назначение: дифференцированная обработка ошибок при работе с файлами.

Классы меню:

Назначение: предоставление пользовательского интерфейса для выбора действий.

Расширение класса Game:

Назначение: добавление возможности сохранения и восстановления

состояния.

Игровой цикл с учётом сохранений

1. Запуск приложения → `GameApplication`
2. Отображение главного меню (`StartMenu`)
3. Выбор действия:

Новая игра → запрос размеров поля → создание `Game` → игровой цикл

Загрузить → `SaveLoadMenu` (список сохранений) → выбор → `loadGame()`

Выход → завершение программы

Обработка исключительных ситуаций

Все операции с файловой системой обёрнуты в try-catch блоки с использованием специализированных исключений:

- При сохранении — перехват `SaveException`, вывод сообщения, возврат к игре.
- При загрузке — перехват `LoadException`, вывод сообщения, возврат в меню.
- При повреждённом файле
— `CorruptedDataException` (наследник `LoadException`) с предложением выбрать другое сохранение.

Результаты тестирования:

Действие	Ожидаемый результат	Результат
Запуск программы	Отображение меню с тремя пунктами	Пройдено
Выбор "Новая игра"	Запрос размеров поля, начало игры	Пройдено
Выбор "Загрузить" при	Сообщение "Нет доступных	Пройдено

отсутствии сохранений	сохранений"	
Выбор "Загрузить" при наличии сохранений	Список файлов с нумерацией	Пройдено
Выбор сохранения из списка	Загрузка игры, продолжение с сохранённого места	Пройдено
Завершение игры, повторный запуск, загрузка	Все параметры восстановлены	Пройдено
Нажатие 'x' во время игры	Сообщение о сохранении, файл в папке saves/	Пройдено

Таблица 1 – тестирование

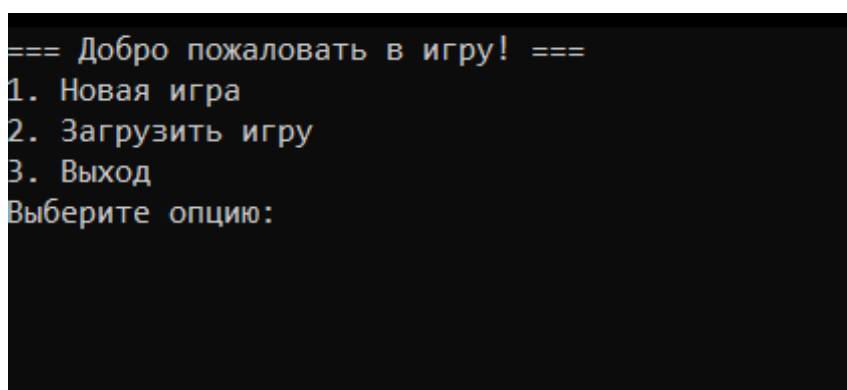


Рисунок 2 – Запуск игры

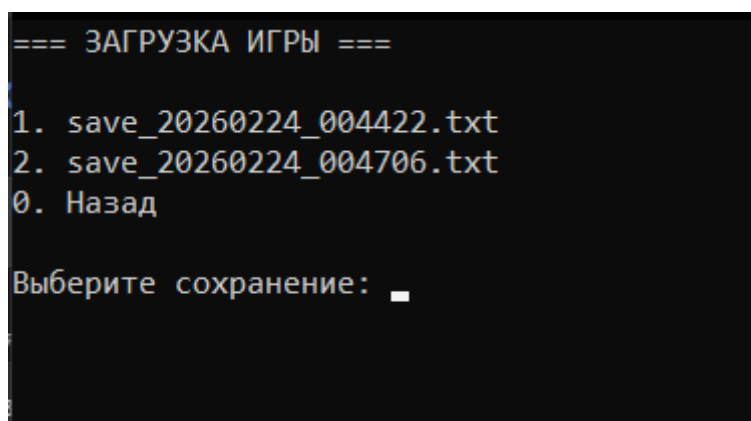


Рисунок 3 – Меню сохранений

```
2 . . . . .
3 . . . . .
4 . . . . .
5 . . . . .

=== Игрок ===
Здоровье: 960 | Урон: 10 | Очки: 10
Врагов осталось: 2
Прогресс убийств: 1/3

--- Рука игрока ---
1. Взрывная волна (радиус: 4)

Команды: w/a/s/d - движение, f - открыть заклинания, x - сохранить, q - выход
Игра сохранена в файл: save_20260224_012621.txt

Игра сохранена!
Нажмите любую клавишу...
```

Рисунок 4 – Сохранение игры

Вывод:

В ходе выполнения лабораторной работы была успешно реализована система сохранения и загрузки состояния игры.

Основные результаты:

1. Архитектурные решения: созданы новые классы с чётким разделением ответственности: SaveManager для файловых операций, StartMenu и SaveLoadMenu для пользовательского интерфейса, GameApplication для координации.
2. Сериализация: реализован текстовый формат с маркерами секций, обеспечивающий читаемость и устойчивость к изменениям.
3. Обработка исключений: создана иерархия классов исключений (SaveException, LoadException), обеспечивающая дифференцированную обработку различных ошибок.
4. Интеграция: существующая игровая логика не изменялась — добавлены только необходимые методы сбора и восстановления

состояния.

5. Пользовательский интерфейс: реализовано полноценное меню с возможностью выбора сохранения из списка.
6. Тестирование: проведена проверка всех ключевых сценариев, включая граничные случаи (отсутствие сохранений, повреждённые файлы).

Разработанная система обеспечивает консистентность сохранений между запусками игры и корректно обрабатывает все исключительные ситуации, предусмотренные заданием.

Приложение

main.cpp:

```
#include "Menu.h"
#include "Constants.h"
#include <cstdlib>
#include <ctime>
#include <windows.h>

int main() {
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
    srand(time(NULL));

    GameApplication app;
    app.run();

    return 0;
}
```

Cell.cpp:

```
#include "Cell.h"

Cell::Cell() : symbol('.'), hasEnemy(false), hasPlayer(false) {}

Cell::Cell(const Cell& other)
    : symbol(other.symbol), hasEnemy(other.hasEnemy),
    hasPlayer(other.hasPlayer) {}

Cell::Cell(Cell&& other) noexcept
    : symbol(other.symbol), hasEnemy(other.hasEnemy),
    hasPlayer(other.hasPlayer) {
    other.symbol = '.';
    other.hasEnemy = false;
    other.hasPlayer = false;
}

Cell& Cell::operator=(const Cell& other) {
```

```

        if (this != &other) {
            symbol = other.symbol;
            hasEnemy = other.hasEnemy;
            hasPlayer = other.hasPlayer;
        }
        return *this;
    }

Cell& Cell::operator=(Cell&& other) noexcept {
    if (this != &other) {
        symbol = other.symbol;
        hasEnemy = other.hasEnemy;
        hasPlayer = other.hasPlayer;
        other.symbol = '.';
        other.hasEnemy = false;
        other.hasPlayer = false;
    }
    return *this;
}

char Cell::getSymbol() const { return symbol; }

bool Cell::isEmpty() const {
    return symbol == '.' && !hasEnemy && !hasPlayer;
}

bool Cell::isEnemy() const { return hasEnemy; }
bool Cell::isPlayer() const { return hasPlayer; }

void Cell::setEnemy(bool present) {
    hasEnemy = present;
    if (present) symbol = 'E';
    else if (!hasPlayer) symbol = '.';
}

void Cell::setPlayer(bool present) {
    hasPlayer = present;
    if (present) symbol = 'P';
}

```

```

        else if (!hasEnemy) symbol = '.';
    }

```

Enemy.cpp:

```

#include "Enemy.h"

```

```

extern const int DEFAULT_ENEMY_HEALTH;
extern const int DEFAULT_ENEMY_DAMAGE;

```

```

Enemy::Enemy() : health(0), damage(0), x(0), y(0), alive(false) {}

```

```

Enemy::Enemy(int startX, int startY)
    : health(DEFAULT_ENEMY_HEALTH), damage(DEFAULT_ENEMY_DAMAGE),
  x(startX), y(startY), alive(true) {}

```

```

Enemy::Enemy(int startX, int startY, int startHealth, int startDamage)
    : health(startHealth), damage(startDamage), x(startX), y(startY),
  alive(true) {}

```

```

Enemy::Enemy(const Enemy& other)
    : health(other.health), damage(other.damage),
  x(other.x), y(other.y), alive(other.alive) {}

```

```

Enemy& Enemy::operator=(const Enemy& other) {
    if (this != &other) {
        health = other.health;
        damage = other.damage;
        x = other.x;
        y = other.y;
        alive = other.alive;
    }
    return *this;
}

```

```

Enemy::Enemy(Enemy&& other) noexcept
    : health(other.health), damage(other.damage),
  x(other.x), y(other.y), alive(other.alive) {
    other.alive = false;
}

```

```

        other.x = -1;
        other.y = -1;
    }

Enemy& Enemy::operator=(Enemy&& other) noexcept {
    if (this != &other) {
        health = other.health;
        damage = other.damage;
        x = other.x;
        y = other.y;
        alive = other.alive;
        other.alive = false;
        other.x = -1;
        other.y = -1;
    }
    return *this;
}

int Enemy::getX() const { return x; }
int Enemy::getY() const { return y; }
bool Enemy::isAlive() const { return alive; }
int Enemy::getDamage() const { return damage; }
int Enemy::getHealth() const { return health; }

void Enemy::setPosition(int newX, int newY) {
    x = newX;
    y = newY;
}

void Enemy::takeDamage(int amount) {
    health -= amount;
    if (health <= 0) {
        alive = false;
    }
}

void Enemy::moveTowardsPlayer(int playerX, int playerY) {
    if (!alive) return;

```

```

        if (playerX > x) x++;
        else if (playerX < x) x--;

        if (playerY > y) y++;
        else if (playerY < y) y--;
    }

```

Game.cpp:

```

#include "Game.h"
#include "Menu.h"
#include "Constants.h"
#include <iostream>
#include <cstdlib>
#include <conio.h>

extern const int SCORE_PER_KILL;
extern const int DEFAULT_ENEMY_DAMAGE;

Game::Game(int width, int height)
    : field(width, height),
      player(width/2, height/2),
      running(true),
      spellMode(false),
      selectingTarget(false),
      selectedSpell(-1),
      targetX(0), targetY(0),
      turnUsed(false) {

    field.placePlayer(player.getX(), player.getY());

    for (int i = 0; i < 3; i++) {
        spawnEnemy();
    }
}

void Game::spawnEnemy() {
    int x, y;

```

```

do {
    x = rand() % field.getWidth();
    y = rand() % field.getHeight();
} while (!field.isCellEmpty(x, y));

enemies.push_back(Enemy(x, y));
field.placeEnemy(x, y);
}

void Game::display() {
    system("cls");
    field.display();

    std::cout << "\n=== Игрок ===\n";
    std::cout << "Здоровье: " << player.getHealth()
        << " | Урон: " << player.getDamage()
        << " | Очки: " << player.getScore() << "\n";
    std::cout << "Врагов осталось: " << getAliveEnemiesCount() <<
"\n";
    std::cout << "Прогресс убийств: " << player.getCurrentKills()
        << "/" << player.getKillsNeeded() << "\n";

    player.getHand()->display();

    if (spellMode) {
        std::cout << "\n=== РЕЖИМ ЗАКЛИНАНИЙ ===\n";
        if (selectingTarget) {
            std::cout << "Цель: (" << targetX << "," << targetY <<
"\n";

            std::cout << "Перемещение: w/a/s/d, Enter - применить, ESC
- отмена\n";
        } else {
            std::cout << "Выберите заклинание (1-" <<
player.getHand()->getCount()
        << ") или ESC для выхода\n";
        }
    } else {
        std::cout << "\nКоманды: w/a/s/d - движение, f - заклинания, x

```

```

- сохранить, q - выход\n";
    }

    if (turnUsed) {
        std::cout << "Ход использован! Ожидайте хода врагов...\n";
    }
}

int Game::getAliveEnemiesCount() {
    int count = 0;
    for (auto& enemy : enemies) {
        if (enemy.isAlive()) count++;
    }
    return count;
}

void Game::handleInput(char key) {
    if (turnUsed) {
        return;
    }

    if (key == 'x' || key == 'X') {
        saveToFile();
        return;
    }

    if (spellMode) {
        handleSpellInput(key);
        return;
    }

    switch(key) {
        case 'w': case 'a': case 's': case 'd':
            handleMovement(key);
            break;
        case 'f':
            enterSpellMode();
            break;
    }
}

```

```

        case 'q':
            running = false;
            break;
    }
}

void Game::handleMovement(char key) {
    int newX = player.getX();
    int newY = player.getY();

    switch(key) {
        case 'w': newY--; break;
        case 's': newY++; break;
        case 'a': newX--; break;
        case 'd': newX++; break;
    }

    if (!field.isValidPosition(newX, newY)) {
        std::cout << "Нельзя выйти за границы поля!\n";
        std::cout << "Нажмите любую клавишу...\n";
        _getch();
        return;
    }

    if (field.isCellEnemy(newX, newY)) {
        attackEnemyAt(newX, newY);
        turnUsed = true;
    }

    else if (field.isCellEmpty(newX, newY)) {
        field.clearCell(player.getX(), player.getY());
        player.setPosition(newX, newY);
        field.placePlayer(player.getX(), player.getY());
        turnUsed = true;
    }
}

void Game::attackEnemyAt(int x, int y) {
    for (auto& enemy : enemies) {

```



```

        if (enemy.isAlive() && enemy.getX() == x && enemy.getY() == y)
        {
            std::cout << "Атака! Нанесен урон врагу!\n";
            enemy.takeDamage(player.getDamage());
            if (!enemy.isAlive()) {
                field.clearCell(x, y);
                player.addScore(SCORE_PER_KILL);
                std::cout << "Враг уничтожен! +" << SCORE_PER_KILL <<
" очков\n";

                if (getAliveEnemiesCount() == 0) {
                    std::cout << "\nВЫ ПОБЕДИЛИ! Все враги
уничтожены!\n";

                    std::cout << "Нажмите любую клавишу...\n";
                    _getch();
                    running = false;
                    return;
                }
            }

            std::cout << "Нажмите любую клавишу...\n";
            _getch();
            break;
        }
    }
}

void Game::enterSpellMode() {
    if (player.getHand()->getCount() == 0) {
        std::cout << "Нет заклинаний для использования!\n";
        std::cout << "Нажмите любую клавишу...\n";
        _getch();
        return;
    }
    spellMode = true;
    selectingTarget = false;
    selectedSpell = -1;
}

```

```

void Game::exitSpellMode() {
    spellMode = false;
    selectingTarget = false;
    selectedSpell = -1;
}

void Game::handleSpellInput(char key) {
    if (!selectingTarget) {
        if (key >= '1' && key <= '9') {
            int index = key - '0';
            if (index <= player.getHand()->getCount()) {
                selectedSpell = index;
                selectingTarget = true;
                targetX = player.getX();
                targetY = player.getY();
            }
        }
        else if (key == 27) {
            exitSpellMode();
        }
    }
    else {
        int newX = targetX;
        int newY = targetY;

        switch(key) {
            case 'w': newY--; break;
            case 's': newY++; break;
            case 'a': newX--; break;
            case 'd': newX++; break;
            case 13:
                if (player.getHand()->useSpell(selectedSpell - 1,
targetX, targetY,
field, enemies, player))
{
                    turnUsed = true;

                    if (getAliveEnemiesCount() == 0) {

```

```

        std::cout << "\nВЫ ПОВЕДИЛИ! Все враги
уничтожены!\n";

        std::cout << "Нажмите любую клавишу...\n";
        _getch();
        running = false;
        return;
    }
}
exitSpellMode();
return;
case 27:
    selectingTarget = false;
    return;
}

if (field.isValidPosition(newX, newY)) {
    targetX = newX;
    targetY = newY;
}
}
}

void Game::enemyTurn() {
    std::cout << "\n--- Ход врагов ---\n";

    for (auto& enemy : enemies) {
        if (!enemy.isAlive()) continue;

        int oldX = enemy.getX();
        int oldY = enemy.getY();

        enemy.moveTowardsPlayer(player.getX(), player.getY());

        int newX = enemy.getX();
        int newY = enemy.getY();

        if (newX == player.getX() && newY == player.getY()) {
            player.takeDamage(enemy.getDamage());
        }
    }
}

```

```

        enemy.setPosition(oldX, oldY);
    }
    else if (field.isCellEmpty(newX, newY)) {
        field.clearCell(oldX, oldY);
        field.placeEnemy(newX, newY);
    } else {
        enemy.setPosition(oldX, oldY);
    }
}

std::cout << "Ход врагов завершен. Нажмите любую клавишу...\n";
_getch();

player.resetTurn();
turnUsed = false;
}

bool Game::isRunning() {
    if (!player.isAlive()) {
        std::cout << "\n=== ИГРОК ПОГИБ ===\n";
        std::cout << "Финальный счет: " << player.getScore() <<
std::endl;
        std::cout << "Нажмите любую клавишу для выхода...\n";
        _getch();
        return false;
    }

    if (getAliveEnemiesCount() == 0 && running) {
        std::cout << "\nВЫ ПОБЕДИЛИ! Все враги уничтожены!\n";
        std::cout << "Нажмите любую клавишу...\n";
        _getch();
        return false;
    }

    return running;
}

bool Game::isTurnUsed() const { return turnUsed; }

```

```

void Game::saveToFile() {
    GameSaveData data = getSaveData();
    SaveManager saver;
    saver.saveGame(data);
    std::cout << "\nИгра сохранена!\n";
    std::cout << "Нажмите любую клавишу...\n";
    _getch();
}

GameSaveData Game::getSaveData() const {
    GameSaveData data;

    data.width = field.getWidth();
    data.height = field.getHeight();

    data.playerX = player.getX();
    data.playerY = player.getY();
    data.playerHealth = player.getHealth();
    data.playerScore = player.getScore();
    data.playerKills = player.getCurrentKills();

    data.spells = player.getSpellNames();

    for (const auto& enemy : enemies) {
        if (enemy.isAlive()) {
            data.enemies.push_back(EnemySaveData(
                enemy.getX(),
                enemy.getY(),
                enemy.getHealth()
            ));
        }
    }

    return data;
}

void Game::loadFromSave(const GameSaveData& data) {

```

```

    field = GameField(data.width, data.height);

    player = Player(data.playerX, data.playerY, data.playerHealth,
                    data.playerScore, data.playerKills);

    player.setSpells(data.spells);

    enemies.clear();
    for (const auto& ed : data.enemies) {
        Enemy enemy(ed.x, ed.y, ed.health, DEFAULT_ENEMY_DAMAGE);
        enemies.push_back(enemy);
        field.placeEnemy(ed.x, ed.y);
    }

    field.placePlayer(data.playerX, data.playerY);

    running = true;
    spellMode = false;
    selectingTarget = false;
    selectedSpell = -1;
    turnUsed = false;
}

void GameLauncher::run() {
    try {
        system("cls");
        std::cout << "Введите размер поля (ширина высота) от "
                    << MIN_FIELD_SIZE << " до " << MAX_FIELD_SIZE << ": ";

        int width, height;
        std::cin >> width >> height;

        Game game(width, height);

        while (game.isRunning()) {
            game.display();

            char key = _getch();

```

```

        game.handleInput(key);

        if (game.isRunning() && game.isTurnUsed()) {
            game.enemyTurn();
        }
    }

} catch (const std::exception& e) {
    std::cout << "Ошибка: " << e.what() << std::endl;
    std::cout << "Нажмите любую клавишу...\n";
    _getch();
}

}

void GameLauncher::loadGame(const std::string& filename) {
    try {
        SaveManager saver;
        GameSaveData data = saver.loadGame(filename);

        Game game(10, 10);
        game.loadFromSave(data);

        while (game.isRunning()) {
            game.display();

            char key = _getch();
            game.handleInput(key);

            if (game.isRunning() && game.isTurnUsed()) {
                game.enemyTurn();
            }
        }

    } catch (const LoadException& e) {
        std::cout << "\nОшибка загрузки: " << e.what() << "\n";
        std::cout << "Нажмите любую клавишу...\n";
        _getch();
    }
}

```

```
}
```

Gamefield.cpp:

```
#include "GameField.h"
```

```
#include <iostream>
```

```
#include <stdexcept>
```

```
extern const int MIN_FIELD_SIZE;
```

```
extern const int MAX_FIELD_SIZE;
```

```
GameField::GameField() : width(0), height(0) {}
```

```
GameField::GameField(int w, int h) : width(w), height(h) {  
    if (w < MIN_FIELD_SIZE || w > MAX_FIELD_SIZE || h < MIN_FIELD_SIZE  
    || h > MAX_FIELD_SIZE) {  
        throw std::invalid_argument("Размер поля должен быть от " +  
std::to_string(MIN_FIELD_SIZE) + " до " +  
std::to_string(MAX_FIELD_SIZE));  
    }  
    grid.resize(height, std::vector<Cell>(width));  
}
```

```
GameField::GameField(const GameField& other)  
    : width(other.width), height(other.height), grid(other.grid) {}
```

```
GameField& GameField::operator=(const GameField& other) {  
    if (this != &other) {  
        width = other.width;  
        height = other.height;  
        grid = other.grid;  
    }  
    return *this;  
}
```

```
GameField::GameField(GameField&& other) noexcept  
    : width(other.width), height(other.height),  
grid(std::move(other.grid)) {  
    other.width = 0;  
    other.height = 0;
```



```

}

GameField& GameField::operator=(GameField&& other) noexcept {
    if (this != &other) {
        width = other.width;
        height = other.height;
        grid = std::move(other.grid);
        other.width = 0;
        other.height = 0;
    }
    return *this;
}

int GameField::getWidth() const { return width; }
int GameField::getHeight() const { return height; }

bool GameField::isValidPosition(int x, int y) const {
    return x >= 0 && x < width && y >= 0 && y < height;
}

void GameField::placePlayer(int x, int y) {
    if (isValidPosition(x, y)) {
        grid[y][x].setPlayer(true);
    }
}

void GameField::placeEnemy(int x, int y) {
    if (isValidPosition(x, y)) {
        grid[y][x].setEnemy(true);
    }
}

void GameField::clearCell(int x, int y) {
    if (isValidPosition(x, y)) {
        grid[y][x].setPlayer(false);
        grid[y][x].setEnemy(false);
    }
}

```

```

void GameField::clearAll() {
    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {
            grid[y][x].setPlayer(false);
            grid[y][x].setEnemy(false);
        }
    }
}

bool GameField::isEmpty(int x, int y) const {
    if (!isValidPosition(x, y)) return false;
    return grid[y][x].isEmpty();
}

bool GameField::isEnemy(int x, int y) const {
    if (!isValidPosition(x, y)) return false;
    return grid[y][x].isEnemy();
}

void GameField::display() const {
    std::cout << "\n ";
    for (int x = 0; x < width; x++) {
        std::cout << x % 10 << " ";
    }
    std::cout << "\n";

    for (int y = 0; y < height; y++) {
        std::cout << y % 10 << " ";
        for (int x = 0; x < width; x++) {
            std::cout << grid[y][x].getSymbol() << " ";
        }
        std::cout << "\n";
    }
}

```

Cell.h:

```

#ifndef CELL_H

```

```

#define CELL_H

class Cell {
private:
    char symbol;
    bool hasEnemy;
    bool hasPlayer;

public:
    Cell();
    Cell(const Cell& other);
    Cell(Cell&& other) noexcept;
    Cell& operator=(const Cell& other);
    Cell& operator=(Cell&& other) noexcept;

    char getSymbol() const;
    bool isEmpty() const;
    bool isEnemy() const;
    bool isPlayer() const;

    void setEnemy(bool present);
    void setPlayer(bool present);
};

#endif

```

Enemy.h:

```

#ifndef ENEMY_H
#define ENEMY_H

class Enemy {
private:
    int health;
    int damage;
    int x, y;
    bool alive;

public:
    Enemy();

```

```

    Enemy(int startX, int startY);
    Enemy(int startX, int startY, int startHealth, int startDamage);
    Enemy(const Enemy& other);
    Enemy(Enemy&& other) noexcept;
    Enemy& operator=(const Enemy& other);
    Enemy& operator=(Enemy&& other) noexcept;

    int getX() const;
    int getY() const;
    bool isAlive() const;
    int getDamage() const;
    int getHealth() const;

    void setPosition(int newX, int newY);
    void takeDamage(int amount);
    void moveTowardsPlayer(int playerX, int playerY);
};

```

```

#endif

```

Game.h:

```

#ifndef GAME_H
#define GAME_H

#include "GameField.h"
#include "Player.h"
#include "Enemy.h"
#include "SaveManager.h"
#include <vector>

class Game {
private:
    GameField field;
    Player player;
    std::vector<Enemy> enemies;
    bool running;
    bool spellMode;
    bool selectingTarget;
    int selectedSpell;

```

```

        int targetX, targetY;
        bool turnUsed;

public:
    Game(int width, int height);

    void spawnEnemy();
    void display();
    int getAliveEnemiesCount();
    void handleInput(char key);
    void handleMovement(char key);
    void attackEnemyAt(int x, int y);
    void enterSpellMode();
    void exitSpellMode();
    void handleSpellInput(char key);
    void enemyTurn();
    bool isRunning();
    bool isTurnUsed() const;

    void saveToFile();
    GameSaveData getSaveData() const;
    void loadFromSave(const GameSaveData& data);
};

```

```

#endif

```

Gamefield.h:

```

#ifndef GAME_FIELD_H
#define GAME_FIELD_H

#include "Cell.h"
#include <vector>

class GameField {
private:
    int width;
    int height;
    std::vector<std::vector<Cell>> grid;

```

```

public:
    GameField();
    GameField(int w, int h);
    GameField(const GameField& other);
    GameField(GameField&& other) noexcept;
    GameField& operator=(const GameField& other);
    GameField& operator=(GameField&& other) noexcept;

    int getWidth() const;
    int getHeight() const;
    bool isValidPosition(int x, int y) const;

    void placePlayer(int x, int y);
    void placeEnemy(int x, int y);
    void clearCell(int x, int y);
    void clearAll();

    bool isEmptyCell(int x, int y) const;
    bool isEnemyCell(int x, int y) const;

    void display() const;
};

#endif

```

spell.h:

```

#ifndef SPELL_H
#define SPELL_H

#include <string>
#include <vector>
#include "GameField.h"

class Player;
class Enemy;

class Spell {
protected:

```

```

        std::string name;
        int range;
        bool usedThisTurn;

public:
    Spell(const std::string& n, int r);
    virtual ~Spell();

    virtual void use(int targetX, int targetY, GameField& field,
                    std::vector<Enemy>& enemies, Player& player) = 0;

    virtual std::string getDescription() const;

    std::string getName() const;
    int getRange() const;
    bool isUsed() const;
    void setUsed(bool used);

    bool canUse(int playerX, int playerY, int targetX, int targetY)
const;
};

#endif

```

Spell.cpp:

```

#include "Spell.h"
#include <cmath>

Spell::Spell(const std::string& n, int r) : name(n), range(r),
usedThisTurn(false) {}

Spell::~Spell() {}

std::string Spell::getDescription() const {
    return name + " (радиус: " + std::to_string(range) + ")";
}

std::string Spell::getName() const { return name; }

```

```

int Spell::getRange() const { return range; }
bool Spell::isUsed() const { return usedThisTurn; }
void Spell::setUsed(bool used) { usedThisTurn = used; }

bool Spell::canUse(int playerX, int playerY, int targetX, int targetY)
const {
    int distance = abs(playerX - targetX) + abs(playerY - targetY);
    return distance <= range;
}
}

```

Damagespell.h:

```

#ifndef DAMAGE_SPELL_H
#define DAMAGE_SPELL_H

#include "Spell.h"

class DamageSpell : public Spell {
private:
    int damage;

public:
    DamageSpell();
    void use(int targetX, int targetY, GameField& field,
             std::vector<Enemy>& enemies, Player& player) override;
};

#endif

```

Damagespell.cpp:

```

#include "DamageSpell.h"
#include "Player.h"
#include "Enemy.h"
#include <iostream>
#include <conio.h>

extern const int SCORE_PER_KILL;
extern const int DAMAGE_SPELL_DAMAGE;

```



```

DamageSpell::DamageSpell() : Spell("Огненная стрела", 3),
damage(DAMAGE_SPELL_DAMAGE) {}

void DamageSpell::use(int targetX, int targetY, GameField& field,
                      std::vector<Enemy>& enemies, Player& player) {
    std::cout << "\n=== ПРИМЕНЕНИЕ ЗАКЛИНАНИЯ ===\n";
    std::cout << "Заклинание: " << name << "\n";
    std::cout << "Цель: (" << targetX << "," << targetY << ")\n";

    if (!field.isCellEnemy(targetX, targetY)) {
        std::cout << "Нет врага в целевой клетке! Заклинание не
сработало.\n";
        std::cout << "Нажмите любую клавишу для продолжения...\n";
        _getch();
        return;
    }

    for (auto& enemy : enemies) {
        if (enemy.isAlive() && enemy.getX() == targetX && enemy.getY()
== targetY) {
            int oldHealth = enemy.getHealth();
            std::cout << "Враг найден! Текущее здоровье: " <<
oldHealth << "\n";
            std::cout << "Наносится " << damage << " урона!\n";

            enemy.takeDamage(damage);

            if (!enemy.isAlive()) {
                std::cout << "Враг уничтожен! +" << SCORE_PER_KILL <<
" очков\n";
                field.clearCell(targetX, targetY);
                player.addScore(SCORE_PER_KILL);
            } else {
                std::cout << "У врага осталось здоровья: " <<
enemy.getHealth() << "\n";
            }
            break;
        }
    }
}

```

```

    }
    usedThisTurn = true;
    std::cout << "=== ЗАКЛИНАНИЕ ПРИМЕНЕНО ===\n";
    std::cout << "Нажмите любую клавишу для продолжения...\n";
    _getch();
}

```

Areaspell.h:

```

#ifndef AREA_SPELL_H
#define AREA_SPELL_H

#include "Spell.h"

class AreaSpell : public Spell {
private:
    int damage;
    int areaSize;

public:
    AreaSpell();
    void use(int targetX, int targetY, GameField& field,
             std::vector<Enemy>& enemies, Player& player) override;
};

#endif

```

Areaspell.cpp:

```

#include "AreaSpell.h"
#include "Player.h"
#include "Enemy.h"
#include <iostream>
#include <conio.h>

extern const int SCORE_PER_KILL;
extern const int AREA_SPELL_DAMAGE;

AreaSpell::AreaSpell() : Spell("Взрывная волна", 4),
damage(AREA_SPELL_DAMAGE), areaSize(2) {}

```

```

void AreaSpell::use(int targetX, int targetY, GameField& field,
                    std::vector<Enemy>& enemies, Player& player) {
    std::cout << "\n=== ПРИМЕНЕНИЕ ЗАКЛИНАНИЯ ===\n";
    std::cout << "Заклинание: " << name << "\n";
    std::cout << "Центр области: (" << targetX << "," << targetY <<
"\n";
    std::cout << "Область поражения: " << areaSize << "x" << areaSize
<< "\n";

    int enemiesHit = 0;
    int totalDamage = 0;

    for (int dx = 0; dx < areaSize; dx++) {
        for (int dy = 0; dy < areaSize; dy++) {
            int checkX = targetX + dx;
            int checkY = targetY + dy;

            if (!field.isValidPosition(checkX, checkY)) continue;

            for (auto& enemy : enemies) {
                if (enemy.isAlive() && enemy.getX() == checkX &&
enemy.getY() == checkY) {
                    enemiesHit++;
                    totalDamage += damage;

                    std::cout << "Враг в клетке (" << checkX << "," <<
checkY
                        << ") получил " << damage << " урона!\n";

                    enemy.takeDamage(damage);

                    if (!enemy.isAlive()) {
                        std::cout << " Враг уничтожен!\n";
                        field.clearCell(checkX, checkY);
                        player.addScore(SCORE_PER_KILL);
                    }
                    break;
                }
            }
        }
    }
}

```

```

        }
    }
}

if (enemiesHit == 0) {
    std::cout << "В области нет врагов. Заклинание потрачено
впустую.\n";
} else {
    std::cout << "Всего поражено врагов: " << enemiesHit << "\n";
    std::cout << "Суммарный урон: " << totalDamage << "\n";
}

usedThisTurn = true;
std::cout << "=== ЗАКЛИНАНИЕ ПРИМЕНЕНО ===\n";
std::cout << "Нажмите любую клавишу для продолжения...\n";
_getch();
}

```

Hand.h:

```

#ifndef HAND_H
#define HAND_H

#include "Spell.h"
#include <vector>

class Hand {
private:
    std::vector<Spell*> spells;
    int maxSize;

public:
    Hand(int size);
    Hand(const Hand& other);
    Hand(Hand&& other) noexcept;
    Hand& operator=(const Hand& other);
    Hand& operator=(Hand&& other) noexcept;
    ~Hand();

```

```

    bool addSpell(Spell* spell);
    void addRandomSpell();

    std::vector<std::string> getSpellNames() const;
    void setSpellsFromNames(const std::vector<std::string>&
spellNames);

    bool useSpell(int index, int targetX, int targetY,
                  GameField& field, std::vector<Enemy>& enemies,
Player& player);

    void display() const;
    void resetTurn();
    int getCount() const;
};

#endif

```

Hand.cpp:

```

#include "Hand.h"
#include "DamageSpell.h"
#include "AreaSpell.h"
#include "Player.h"
#include "Enemy.h"
#include <iostream>
#include <cstdlib>
#include <conio.h>

Hand::Hand(int size) : maxSize(size) {
    addRandomSpell();
}

Hand::Hand(const Hand& other) : maxSize(other.maxSize) {
    for (Spell* spell : other.spells) {
        if (dynamic_cast<DamageSpell*>(spell)) {
            spells.push_back(new
DamageSpell(*dynamic_cast<DamageSpell*>(spell)));

```

```

        } else if (dynamic_cast<AreaSpell*>(spell)) {
            spells.push_back(new
AreaSpell(*dynamic_cast<AreaSpell*>(spell)));
        }
    }
}

Hand& Hand::operator=(const Hand& other) {
    if (this != &other) {
        for (Spell* spell : spells) {
            delete spell;
        }
        spells.clear();

        maxSize = other.maxSize;
        for (Spell* spell : other.spells) {
            if (dynamic_cast<DamageSpell*>(spell)) {
                spells.push_back(new
DamageSpell(*dynamic_cast<DamageSpell*>(spell)));
            } else if (dynamic_cast<AreaSpell*>(spell)) {
                spells.push_back(new
AreaSpell(*dynamic_cast<AreaSpell*>(spell)));
            }
        }
    }
    return *this;
}

Hand::Hand(Hand&& other) noexcept
    : spells(std::move(other.spells)), maxSize(other.maxSize) {}

Hand& Hand::operator=(Hand&& other) noexcept {
    if (this != &other) {
        spells = std::move(other.spells);
        maxSize = other.maxSize;
    }
    return *this;
}

```

```

Hand::~~Hand() {
    for (Spell* spell : spells) {
        delete spell;
    }
}

bool Hand::addSpell(Spell* spell) {
    if (spells.size() >= maxSize) {
        std::cout << "Рука полна! Нельзя добавить новое
заклинание.\n";
        delete spell;
        return false;
    }

    spells.push_back(spell);
    std::cout << "Добавлено заклинание: " << spell->getName() << "\n";
    std::cout << "Заклинаний в руке: " << spells.size() << "/" <<
maxSize << "\n";
    return true;
}

void Hand::addRandomSpell() {
    if (spells.size() >= maxSize) return;

    int type = rand() % 2;
    if (type == 0) {
        addSpell(new DamageSpell());
    } else {
        addSpell(new AreaSpell());
    }
}

std::vector<std::string> Hand::getSpellNames() const {
    std::vector<std::string> names;
    for (const auto& spell : spells) {
        names.push_back(spell->getName());
    }
}

```

```

        return names;
    }

void Hand::setSpellsFromNames(const std::vector<std::string>&
spellNames) {
    for (Spell* spell : spells) {
        delete spell;
    }
    spells.clear();

    for (const auto& name : spellNames) {
        if (name == "Огненная стрела") {
            spells.push_back(new DamageSpell());
        } else if (name == "Взрывная волна") {
            spells.push_back(new AreaSpell());
        }
    }
}

bool Hand::useSpell(int index, int targetX, int targetY,
                    GameField& field, std::vector<Enemy>& enemies,
Player& player) {
    if (index < 0 || index >= spells.size()) {
        std::cout << "Неверный индекс заклинания!\n";
        std::cout << "Нажмите любую клавишу для продолжения...\n";
        _getch();
        return false;
    }

    Spell* spell = spells[index];

    if (spell->isUsed()) {
        std::cout << "Это заклинание уже использовано в этот ход!\n";
        std::cout << "Нажмите любую клавишу для продолжения...\n";
        _getch();
        return false;
    }
}

```



```

int playerX = player.getX();
int playerY = player.getY();

std::cout << "\nПроверка дальности:\n";
std::cout << "Игрок: (" << playerX << ", " << playerY << ")\n";
std::cout << "Цель: (" << targetX << ", " << targetY << ")\n";

if (!spell->canUse(playerX, playerY, targetX, targetY)) {
    int distance = abs(playerX - targetX) + abs(playerY -
targetY);
    std::cout << "Цель вне радиуса! Расстояние: " << distance
        << ", радиус заклинания: " << spell->getRange() << "\n";
    std::cout << "Нажмите любую клавишу для продолжения...\n";
    _getch();
    return false;
}

std::cout << "Цель в радиусе действия\n";

spell->use(targetX, targetY, field, enemies, player);
return true;
}

void Hand::display() const {
    if (spells.empty()) {
        std::cout << "Рука пуста.\n";
        return;
    }

    std::cout << "\n--- Рука игрока ---\n";
    for (size_t i = 0; i < spells.size(); i++) {
        std::cout << i + 1 << ". " << spells[i]->getDescription();
        if (spells[i]->isUsed()) {
            std::cout << " [ИСПОЛЬЗОВАНО]";
        }
        std::cout << "\n";
    }
}

```

```

void Hand::resetTurn() {
    for (auto spell : spells) {
        spell->setUsed(false);
    }
}

int Hand::getCount() const { return spells.size(); }

```

Player.h:

```

#ifndef PLAYER_H
#define PLAYER_H

#include "Hand.h"
#include <vector>
#include <string>

class Player {
private:
    int health;
    int damage;
    int score;
    int x, y;
    Hand* hand;
    int killsForNewSpell;
    int currentKills;

public:
    Player();
    Player(int startX, int startY);
    Player(int startX, int startY, int startHealth, int startScore,
int startKills);
    Player(const Player& other);
    Player(Player&& other) noexcept;
    Player& operator=(const Player& other);
    Player& operator=(Player&& other) noexcept;
    ~Player();

```

```

    int getX() const;
    int getY() const;
    int getHealth() const;
    int getDamage() const;
    int getScore() const;
    Hand* getHand() const;
    int getCurrentKills() const;
    int getKillsNeeded() const;

    void setPosition(int newX, int newY);
    void takeDamage(int amount);
    void addScore(int points);
    bool isAlive() const;

    void onEnemyKilled();
    void resetTurn();

    std::vector<std::string> getSpellNames() const;
    void setSpells(const std::vector<std::string>& spellNames);
};

#endif

```

Player.cpp:

```

#include "Player.h"
#include <iostream>

extern const int DEFAULT_PLAYER_HEALTH;
extern const int DEFAULT_PLAYER_DAMAGE;
extern const int HAND_MAX_SIZE;
extern const int KILLS_FOR_NEW_SPELL;

Player::Player() : hand(nullptr) {}

Player::Player(int startX, int startY)
    : health(DEFAULT_PLAYER_HEALTH), damage(DEFAULT_PLAYER_DAMAGE),
    score(0), x(startX), y(startY),
    hand(new Hand(HAND_MAX_SIZE)),

```

```
killsForNewSpell(KILLS_FOR_NEW_SPELL), currentKills(0) {}
```

```
Player::Player(int startX, int startY, int startHealth, int  
startScore, int startKills)
```

```
    : health(startHealth), damage(DEFAULT_PLAYER_DAMAGE),  
      score(startScore), x(startX), y(startY),  
      hand(new Hand(HAND_MAX_SIZE)),  
      killsForNewSpell(KILLS_FOR_NEW_SPELL),  
      currentKills(startKills) {}
```

```
Player::Player(const Player& other)
```

```
    : health(other.health), damage(other.damage),  
      score(other.score), x(other.x), y(other.y),  
      hand(new Hand(*other.hand)),  
      killsForNewSpell(other.killsForNewSpell),  
      currentKills(other.currentKills) {}
```

```
Player& Player::operator=(const Player& other) {
```

```
    if (this != &other) {  
        health = other.health;  
        damage = other.damage;  
        score = other.score;  
        x = other.x;  
        y = other.y;  
  
        delete hand;  
        hand = new Hand(*other.hand);  
  
        killsForNewSpell = other.killsForNewSpell;  
        currentKills = other.currentKills;  
    }  
    return *this;  
}
```

```
Player::Player(Player&& other) noexcept
```

```
    : health(other.health), damage(other.damage),  
      score(other.score), x(other.x), y(other.y),  
      hand(other.hand),
```

```

        killsForNewSpell(other.killsForNewSpell),
        currentKills(other.currentKills) {
other.health = 0;
other.x = -1;
other.y = -1;
other.hand = nullptr;
}

Player& Player::operator=(Player&& other) noexcept {
    if (this != &other) {
        health = other.health;
        damage = other.damage;
        score = other.score;
        x = other.x;
        y = other.y;

        delete hand;
        hand = other.hand;

        killsForNewSpell = other.killsForNewSpell;
        currentKills = other.currentKills;

        other.health = 0;
        other.x = -1;
        other.y = -1;
        other.hand = nullptr;
    }
    return *this;
}

Player::~~Player() {
    delete hand;
}

int Player::getX() const { return x; }
int Player::getY() const { return y; }
int Player::getHealth() const { return health; }
int Player::getDamage() const { return damage; }

```

```

int Player::getScore() const { return score; }
Hand* Player::getHand() const { return hand; }
int Player::getCurrentKills() const { return currentKills; }
int Player::getKillsNeeded() const { return killsForNewSpell; }

void Player::setPosition(int newX, int newY) {
    x = newX;
    y = newY;
}

void Player::takeDamage(int amount) {
    health -= amount;
    if (health < 0) health = 0;
    std::cout << "Игрок получил " << amount << " урона! Здоровье: " <<
health << std::endl;
}

void Player::addScore(int points) {
    score += points;
    std::cout << "Получено очков: " << points << ". Всего: " << score
<< "\n";
    onEnemyKilled();
}

bool Player::isAlive() const { return health > 0; }

void Player::onEnemyKilled() {
    currentKills++;
    std::cout << "Прогресс убийств: " << currentKills << "/" <<
killsForNewSpell << "\n";

    if (currentKills >= killsForNewSpell) {
        std::cout << "=== ПОЛУЧЕНО НОВОЕ ЗАКЛИНАНИЕ! ===\n";
        hand->addRandomSpell();
        currentKills = 0;
        std::cout << "Счетчик убийств сброшен. До следующего
заклинания: "
        << killsForNewSpell << " убийств\n";
    }
}

```

```

    }
}

void Player::resetTurn() {
    hand->resetTurn();
}

std::vector<std::string> Player::getSpellNames() const {
    return hand->getSpellNames();
}

void Player::setSpells(const std::vector<std::string>& spellNames) {
    hand->setSpellsFromNames(spellNames);
}

```

SaveManager.h

```

#ifndef SAVE_MANAGER_H
#define SAVE_MANAGER_H

#include <string>
#include <vector>
#include <exception>

struct EnemySaveData {
    int x, y;
    int health;

    EnemySaveData();
    EnemySaveData(int _x, int _y, int h);
};

struct GameSaveData {
    int width, height;
    int playerX, playerY;
    int playerHealth, playerScore, playerKills;
    std::vector<std::string> spells;
    std::vector<EnemySaveData> enemies;
}

```

```

        GameSaveData();
};

class SaveException : public std::exception {
private:
    std::string message;
public:
    SaveException(const std::string& msg);
    const char* what() const noexcept override;
};

class LoadException : public std::exception {
private:
    std::string message;
public:
    LoadException(const std::string& msg);
    const char* what() const noexcept override;
};

class SaveManager {
private:
    std::string saveDir;

    void ensureDirectoryExists();
    std::string getCurrentTimestamp();

public:
    SaveManager(const std::string& dir = "saves/");

    void saveGame(const GameSaveData& data, const std::string&
filename);
    void saveGame(const GameSaveData& data);
    GameSaveData loadGame(const std::string& filename);
    std::vector<std::string> listSaves();
};

#endif

```


SaveManager.cpp

```
#include "SaveManager.h"
#include <fstream>
#include <sstream>
#include <filesystem>
#include <ctime>
#include <iostream>

namespace fs = std::filesystem;

EnemySaveData::EnemySaveData() : x(0), y(0), health(0) {}
EnemySaveData::EnemySaveData(int _x, int _y, int h) : x(_x), y(_y),
health(h) {}

GameSaveData::GameSaveData() : width(0), height(0), playerX(0),
playerY(0),
                                playerHealth(0), playerScore(0),
playerKills(0) {}

SaveException::SaveException(const std::string& msg) : message("Ошибка
сохранения: " + msg) {}
const char* SaveException::what() const noexcept { return
message.c_str(); }

LoadException::LoadException(const std::string& msg) : message("Ошибка
загрузки: " + msg) {}
const char* LoadException::what() const noexcept { return
message.c_str(); }

SaveManager::SaveManager(const std::string& dir) : saveDir(dir) {
    ensureDirectoryExists();
}

void SaveManager::ensureDirectoryExists() {
    if (!fs::exists(saveDir)) {
        fs::create_directory(saveDir);
    }
}
```

```

}

std::string SaveManager::getCurrentTimestamp() {
    time_t now = time(nullptr);
    char buf[20];
    strftime(buf, sizeof(buf), "%Y%m%d_%H%M%S", localtime(&now));
    return std::string(buf);
}

void SaveManager::saveGame(const GameSaveData& data, const
std::string& filename) {
    std::string fullPath = saveDir + filename;
    std::ofstream file(fullPath);

    if (!file.is_open()) {
        throw SaveException("Не удалось создать файл");
    }

    file << "[FIELD]\n";
    file << data.width << " " << data.height << "\n";

    file << "[PLAYER]\n";
    file << data.playerX << " " << data.playerY << " "
        << data.playerHealth << " " << data.playerScore << " "
        << data.playerKills << "\n";

    file << "[SPELLS]\n";
    file << data.spells.size() << "\n";
    for (const auto& spell : data.spells) {
        file << spell << "\n";
    }

    file << "[ENEMIES]\n";
    file << data.enemies.size() << "\n";
    for (const auto& enemy : data.enemies) {
        file << enemy.x << " " << enemy.y << " " << enemy.health <<
"\n";
    }
}

```

```

        file.close();
    }

void SaveManager::saveGame(const GameSaveData& data) {
    std::string filename = "save_" + getCurrentTimestamp() + ".txt";
    saveGame(data, filename);
    std::cout << "Игра сохранена в файл: " << filename << "\n";
}

GameSaveData SaveManager::loadGame(const std::string& filename) {
    std::string fullPath = saveDir + filename;
    std::ifstream file(fullPath);

    if (!file.is_open()) {
        throw LoadException("Файл сохранения не найден");
    }

    GameSaveData data;
    std::string marker;

    file >> marker;
    if (marker != "[FIELD]") throw LoadException("Неверный формат файла");
    file >> data.width >> data.height;

    file >> marker;
    if (marker != "[PLAYER]") throw LoadException("Неверный формат файла");
    file >> data.playerX >> data.playerY
        >> data.playerHealth >> data.playerScore >> data.playerKills;

    file >> marker;
    if (marker != "[SPELLS]") throw LoadException("Неверный формат файла");
    int spellCount;
    file >> spellCount;
    data.spells.clear();
}

```

```

file.ignore();
for (int i = 0; i < spellCount; i++) {
    std::string spell;
    getline(file, spell);
    data.spells.push_back(spell);
}

file >> marker;
if (marker != "[ENEMIES]") throw LoadException("Неверный формат
файла");
int enemyCount;
file >> enemyCount;
data.enemies.clear();
for (int i = 0; i < enemyCount; i++) {
    int x, y, health;
    file >> x >> y >> health;
    data.enemies.push_back(EnemySaveData(x, y, health));
}

return data;
}

std::vector<std::string> SaveManager::listSaves() {
    std::vector<std::string> saves;

    try {
        for (const auto& entry : fs::directory_iterator(saveDir)) {
            if (entry.path().extension() == ".txt") {
                saves.push_back(entry.path().filename().string());
            }
        }
    } catch (...) {}

    return saves;
}

```

Menu.h

```

#ifndef MENU_H

```

```

#define MENU_H

#include <string>
#include <vector>

class StartMenu {
public:
    int run();

private:
    void display();
    int getChoice();
};

class SaveLoadMenu {
public:
    int run();

private:
    void display(const std::vector<std::string>& saves);
    int getChoice(int maxSaves);
};

class GameLauncher {
public:
    void run();
    void loadGame(const std::string& filename);
};

class GameApplication {
private:
    StartMenu menu;
    GameLauncher launcher;

public:
    void run();
};

```

```
#endif
```

Menu.cpp

```
#include "Menu.h"
#include "SaveManager.h"
#include "Game.h"
#include <iostream>
#include <conio.h>
#include <windows.h>

extern const int MIN_FIELD_SIZE;
extern const int MAX_FIELD_SIZE;

int StartMenu::run() {
    while (true) {
        display();
        int choice = getChoice();

        if (choice == 1 || choice == 2 || choice == 3) {
            return choice;
        }
    }
}

void StartMenu::display() {
    system("cls");
    std::cout << "=== Добро пожаловать в игру! ===\n";
    std::cout << "1. Новая игра\n";
    std::cout << "2. Загрузить игру\n";
    std::cout << "3. Выход\n";
    std::cout << "Выберите опцию: ";
}

int StartMenu::getChoice() {
    char choice = _getch();
    while (choice < '1' || choice > '3') {
        std::cout << "\nНеверный выбор! Пожалуйста, выберите 1, 2 или
3: ";
    }
}
```

```

        choice = _getch();
    }
    return choice - '0';
}

int SaveLoadMenu::run() {
    SaveManager saver;
    auto saves = saver.listSaves();

    if (saves.empty()) {
        std::cout << "\nНет доступных сохранений!\n";
        std::cout << "Нажмите любую клавишу...\n";
        _getch();
        return -1;
    }

    while (true) {
        display(saves);
        int choice = getChoice(saves.size());

        if (choice == 0) return -1;
        if (choice > 0) return choice - 1;
    }
}

void SaveLoadMenu::display(const std::vector<std::string>& saves) {
    system("cls");
    std::cout << "=== ЗАГРУЗКА ИГРЫ ===\n\n";

    for (size_t i = 0; i < saves.size(); i++) {
        std::cout << i + 1 << ". " << saves[i] << "\n";
    }

    std::cout << "0. Назад\n";
    std::cout << "\nВыберите сохранение: ";
}

int SaveLoadMenu::getChoice(int maxSaves) {
    char c = _getch();

```

```

        if (c == '0') return 0;

        int choice = c - '0';
        if (choice >= 1 && choice <= maxSaves) {
            return choice;
        }
        return -2;
    }

void GameApplication::run() {
    while (true) {
        int choice = menu.run();

        if (choice == 1) {
            launcher.run();
        }
        else if (choice == 2) {
            SaveLoadMenu loadMenu;
            int saveIndex = loadMenu.run();

            if (saveIndex >= 0) {
                SaveManager saver;
                auto saves = saver.listSaves();
                if (saveIndex < (int)saves.size()) {
                    launcher.loadGame(saves[saveIndex]);
                }
            }
        }
        else if (choice == 3) {
            break;
        }
    }
}

```

Constants.h

```

#ifndef CONSTANTS_H
#define CONSTANTS_H

```



```
extern const int DEFAULT_PLAYER_HEALTH;
extern const int DEFAULT_PLAYER_DAMAGE;
extern const int DEFAULT_ENEMY_HEALTH;
extern const int DEFAULT_ENEMY_DAMAGE;
extern const int KILLS_FOR_NEW_SPELL;
extern const int HAND_MAX_SIZE;
extern const int MIN_FIELD_SIZE;
extern const int MAX_FIELD_SIZE;
extern const int SCORE_PER_KILL;
extern const int DAMAGE_SPELL_DAMAGE;
extern const int AREA_SPELL_DAMAGE;

#endif
```

Constants.cpp

```
#include "Constants.h"

const int DEFAULT_PLAYER_HEALTH = 1000;
const int DEFAULT_PLAYER_DAMAGE = 10;
const int DEFAULT_ENEMY_HEALTH = 30;
const int DEFAULT_ENEMY_DAMAGE = 10;
const int KILLS_FOR_NEW_SPELL = 3;
const int HAND_MAX_SIZE = 5;
const int MIN_FIELD_SIZE = 10;
const int MAX_FIELD_SIZE = 25;
const int SCORE_PER_KILL = 10;
const int DAMAGE_SPELL_DAMAGE = 25;
const int AREA_SPELL_DAMAGE = 15;
```